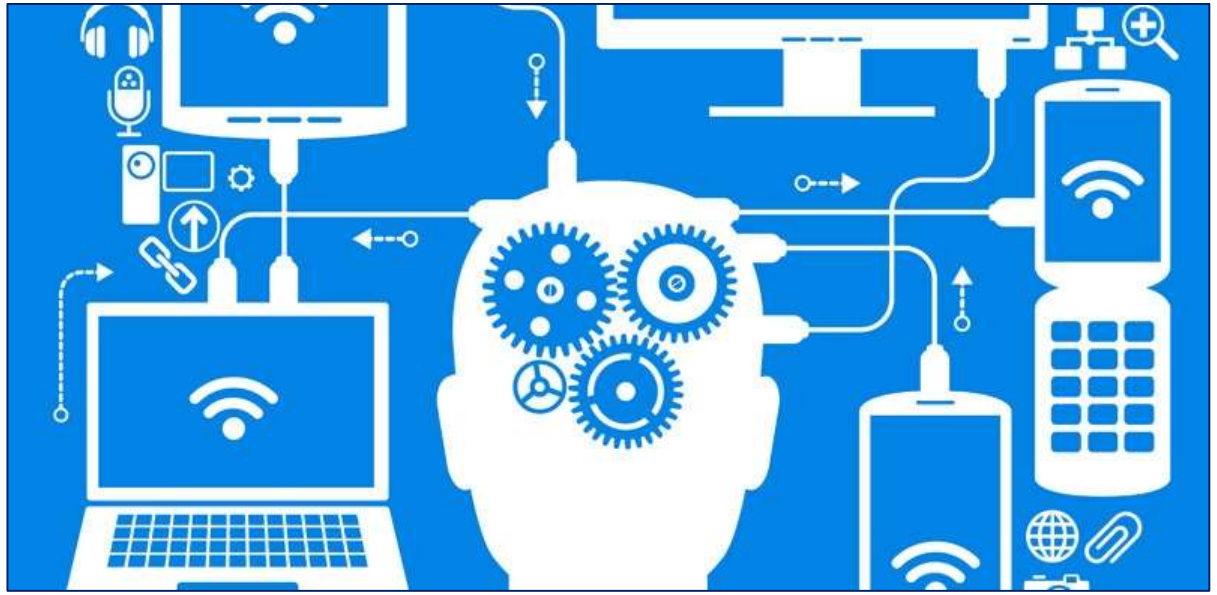
UNIVERSITY of DERBY

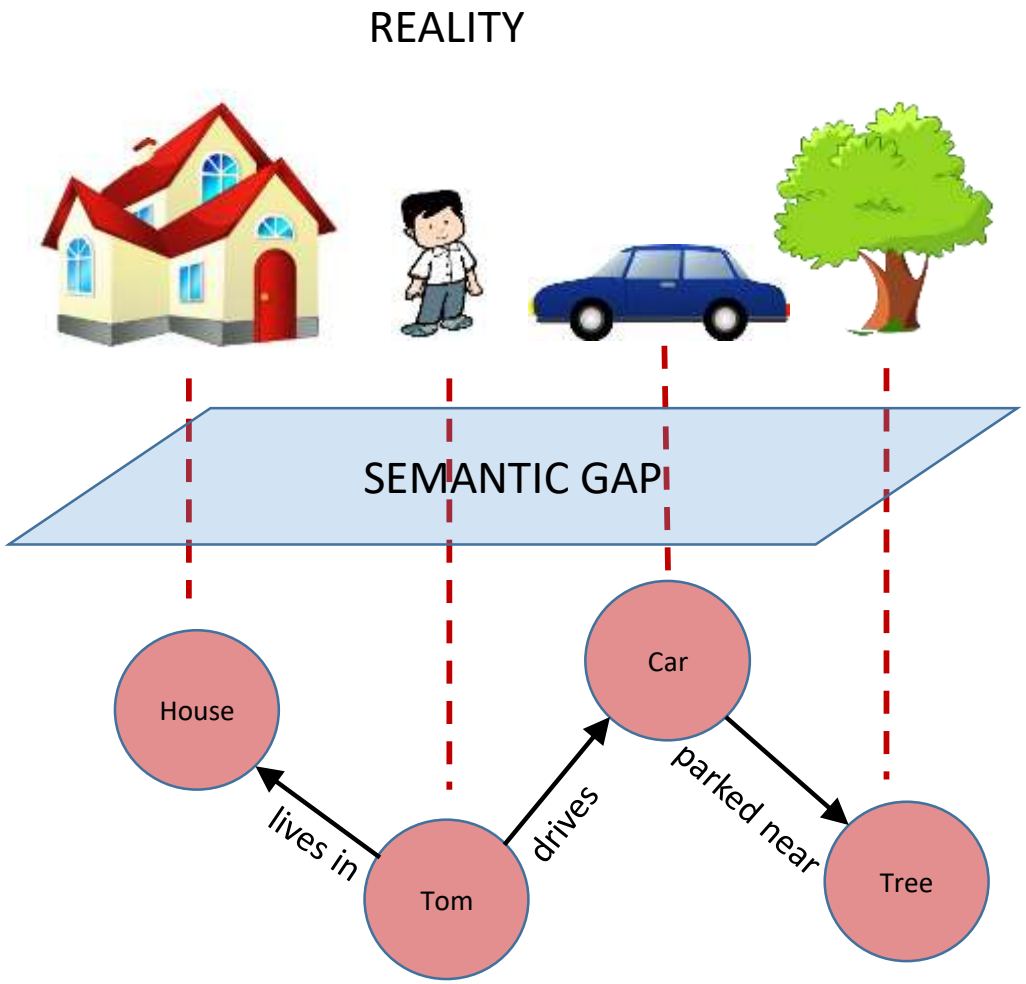
Software Engineering - 5CM505

Lecture 6
Functional Modelling
Object Oriented Development



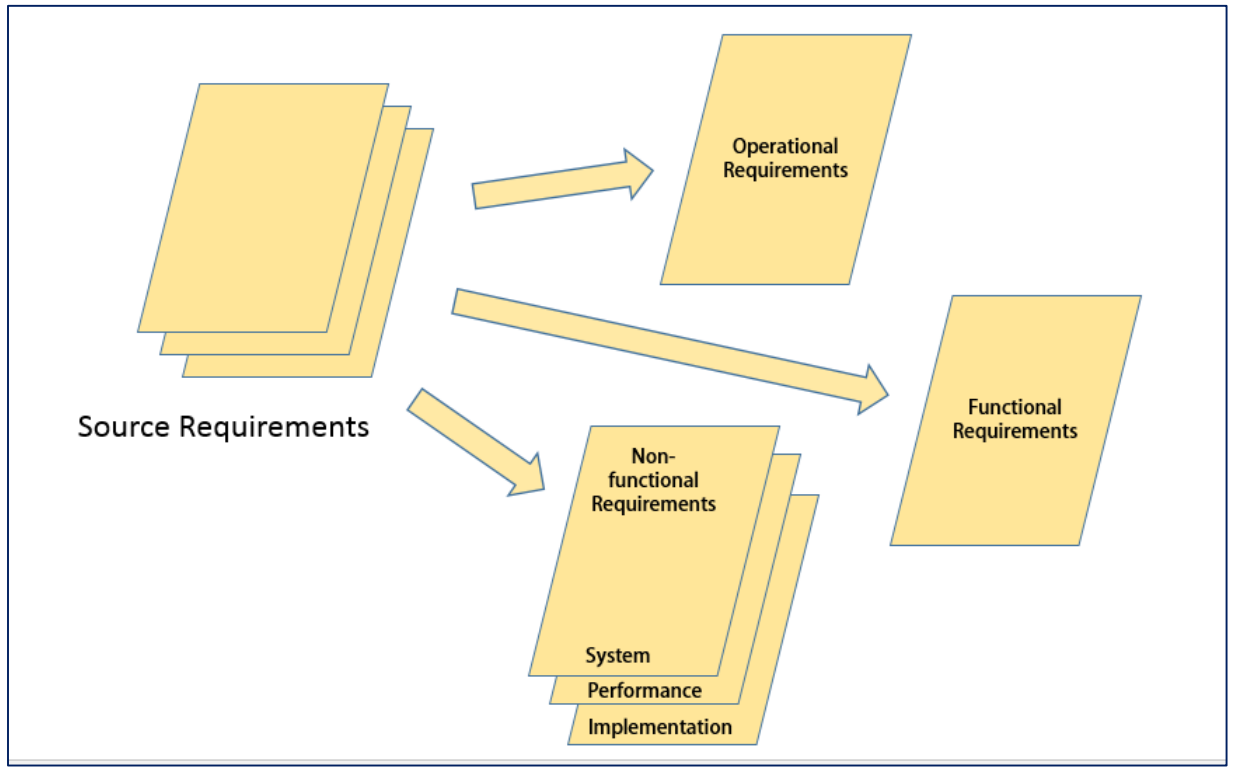
Lecture 6 - Content

- Recap from last week
- Functional Modelling
- System Development Methods
  - Functional / data oriented
  - Object-oriented
- Object-oriented Analysis
- Object-oriented Construction



Recap from Last Week

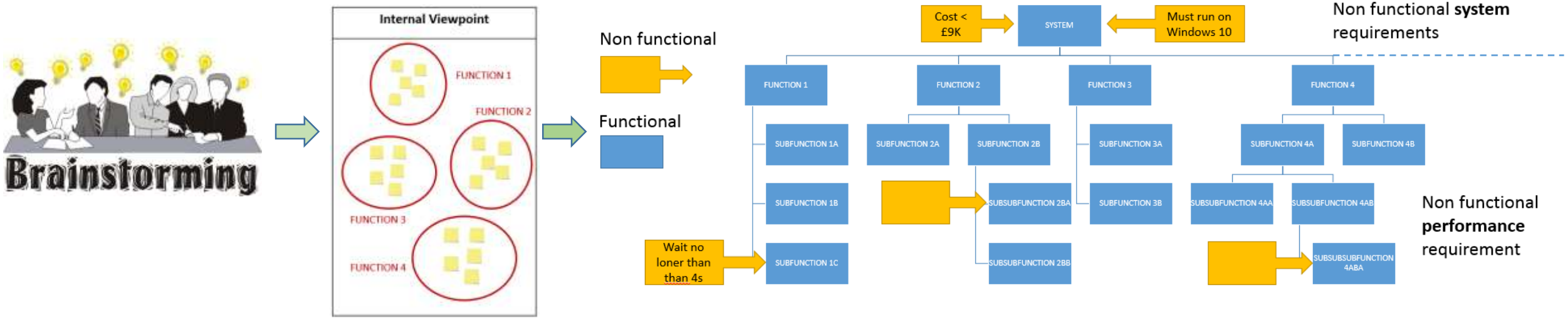
Systemic Textual Analysis



Systemic Textual Analysis			
Project:			Date:
Author:			Issue:
Requirements			Comments
Context:			
Operational Requirement:			
Non-functional System Requirements:			
Non-functional Implementation Requirement	Functional Requirement	Non-functional Performance Requirement	

- Allows us to identify missing and ambiguous requirements
- No relationships between the different functional requirements

Viewpoint Analysis



- Allows us to identify missing and ambiguous requirements
- Relationships between functions and sub-functions **but not between top level functions**

1. Brainstorm
2. Separate into functional / non-functional
3. Separate into internal / external
4. Group functional requirements and name
5. Draw groups as a hierarchical chart
6. Add in the non-functional requirements

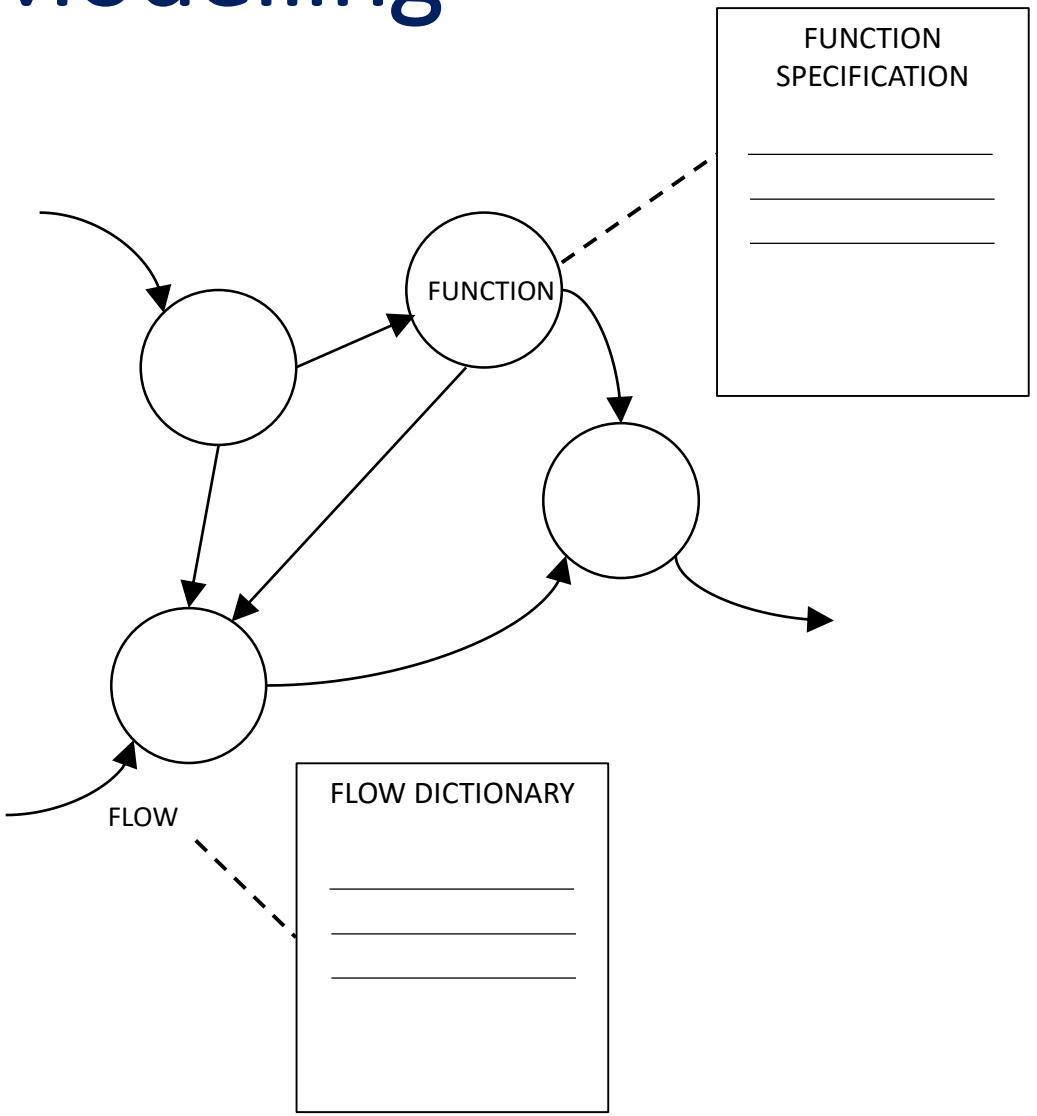
Learning Outcomes

- By the end of this lecture you should be able to:
  - Explain the principles behind functional modelling, discuss its advantages and disadvantages, and use it to aid with designing software systems
  - Describe the differences between functional (data) modelling and object-oriented modelling
  - Discuss the advantages of object-oriented modelling and analysis
  - Explain the OO principles of *encapsulation*, *information hiding*, *inheritance* and *polymorphism*

Functional Modelling

Functional Modelling

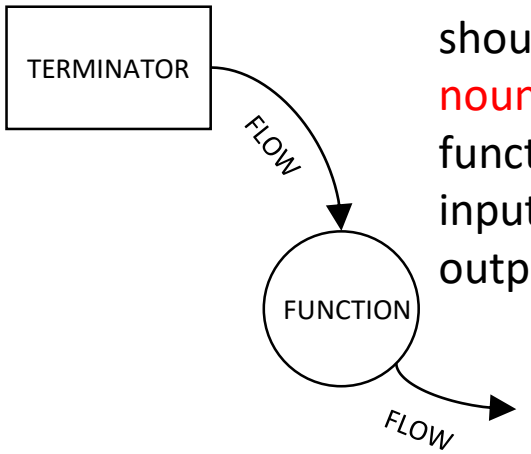
- A tool that highlights the functionality and the inputs and outputs to the identified functions
- Uses three sub-tools:
  - Function Flow Diagram (FFD)
  - Flow Dictionary
  - Function Specification



Function Flow Diagram (FFD)

- An abstract network representation of the system
- Depicts the system in terms of its component functions and the interdependencies or “flows” between them
- Limited diagramming conventions. Consists of only FUNCTIONS, TERMINATORS and FLOWS

Terminator name can be either a verb-noun phrase or a noun. A terminator represents the source or destination of flows **external** to the system



Function name should be a verb-noun phrase. The function transforms input flows into output flows

Flow name should be a noun. The flow represents an input or output

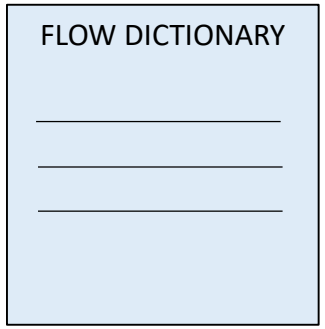
Flow Dictionary and Functional Specification

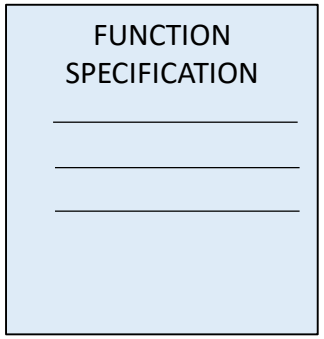
FLOW DICTIONARY (FD)

- Documents each of the flows on the diagram
- Declares the component elements of each input or output flow
- Also declares the relationships between them
- Specifies precisely what is meant by each flow on the FFD

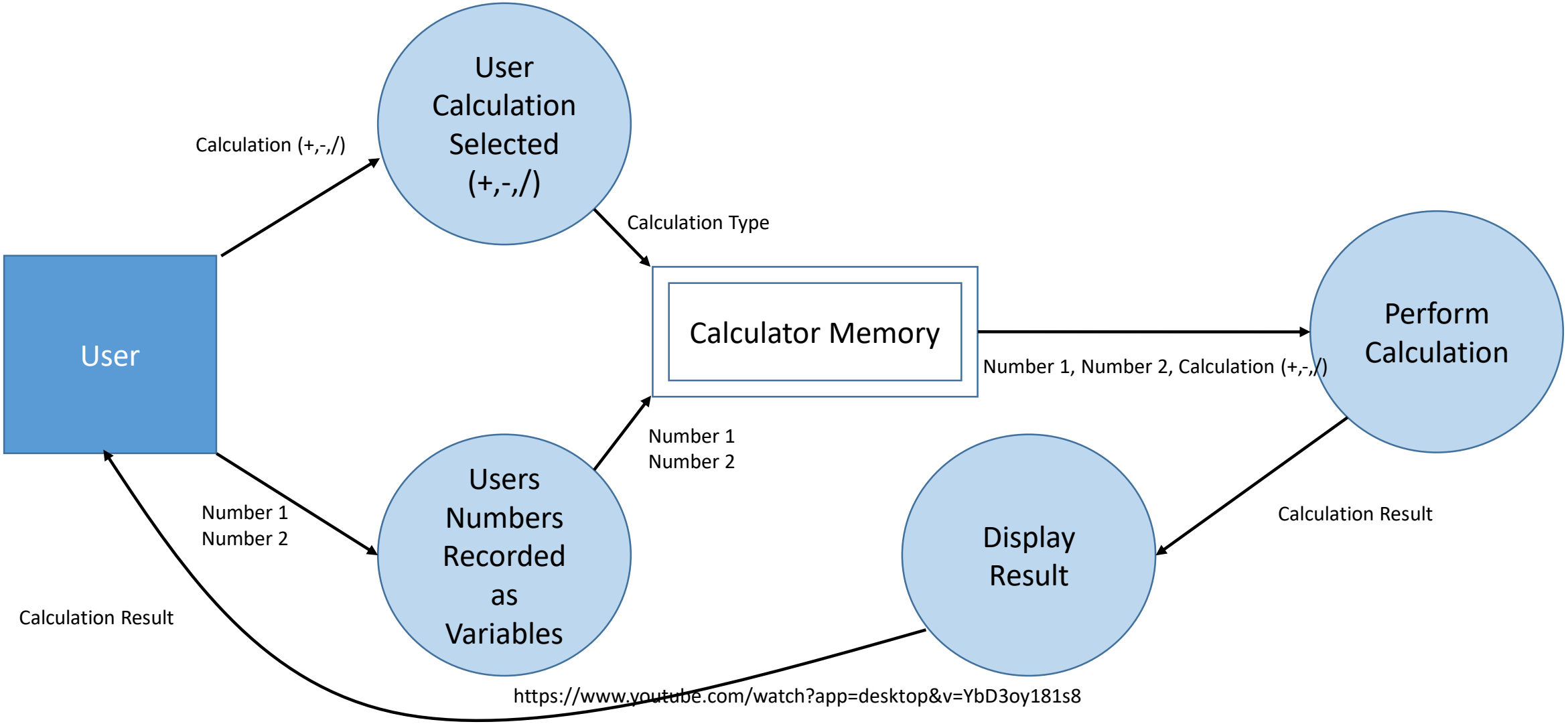
FUNCTIONAL SPECIFICATION (FS)

- Specifies the functions by defining the transformation that converts inputs to outputs



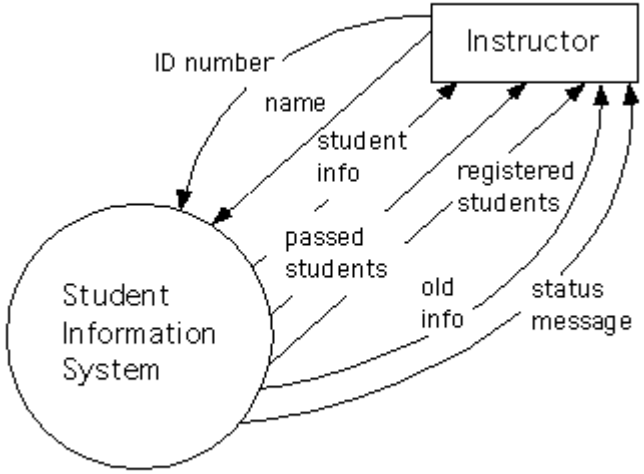


Functional Flow Diagram – Calculator Example

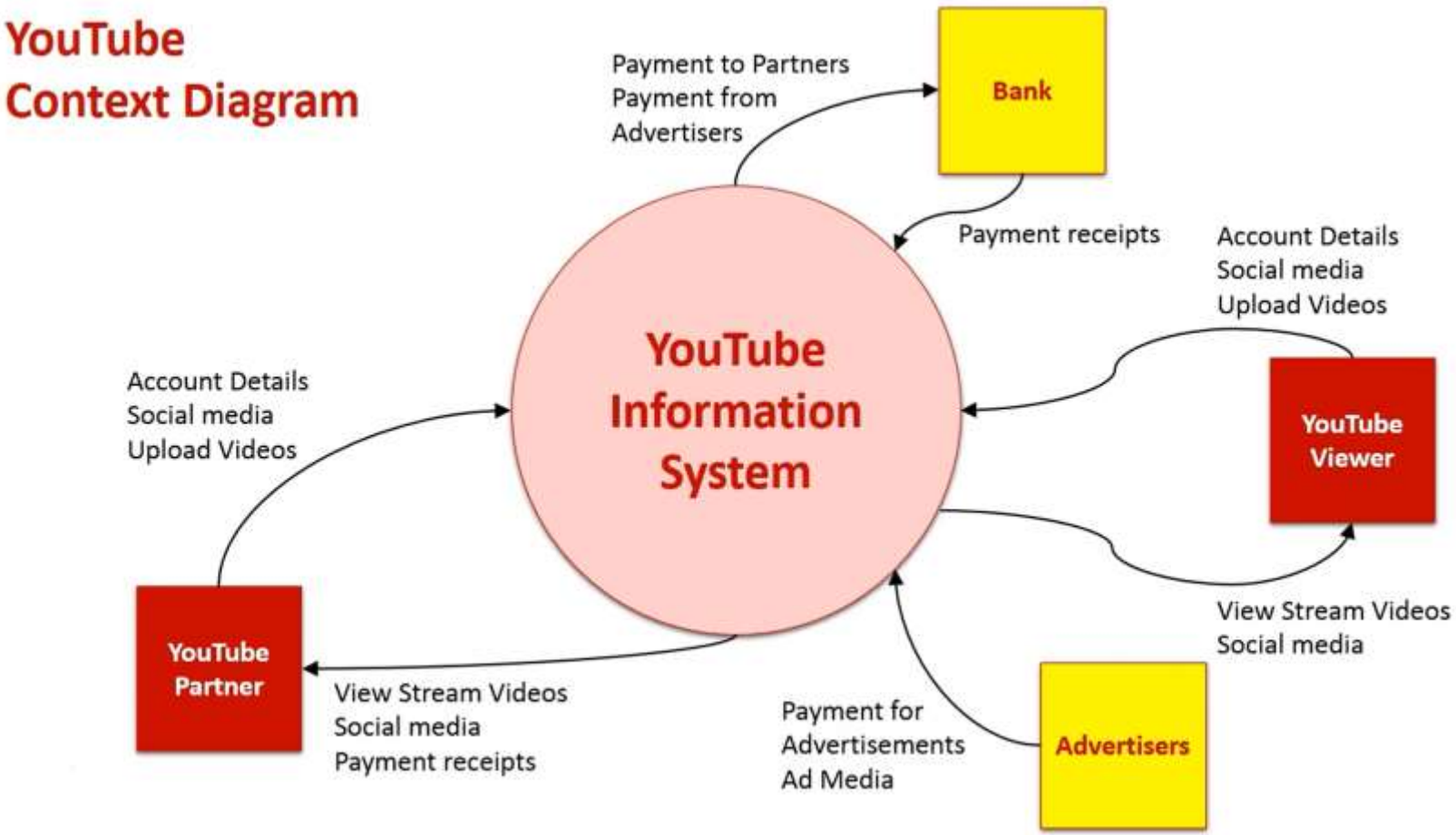


Context Diagram

- For large systems often a set of FFDs is required
- Can start with the top level FFD, which is referred to as the *Context Diagram*
- The Context Diagram shows the system in its operational context – the "super-system". It has only one function
- It can be decomposed into a number of functions that can be represented together on one diagram. This is called DIAGRAM 0. Rule of thumb – no more than 7 functions in one diagram
- The functions in DIAGRAM 0 can be taken in turn and decomposed further into lower-level FFDs
- Can have a hierarchy of FFDs, each level providing lower level detail for the functions in the level directly above

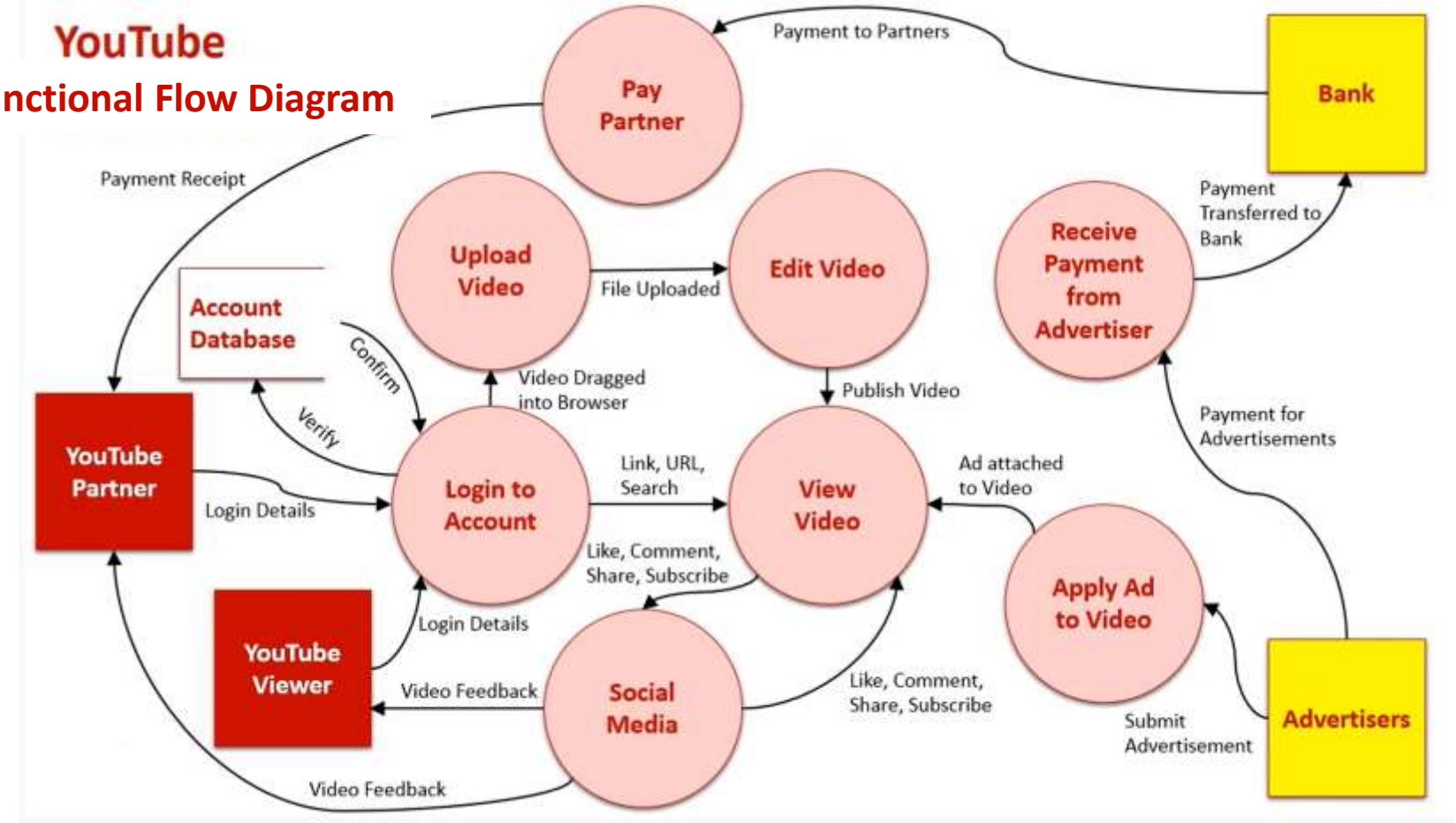


Context Diagram – YouTube Example



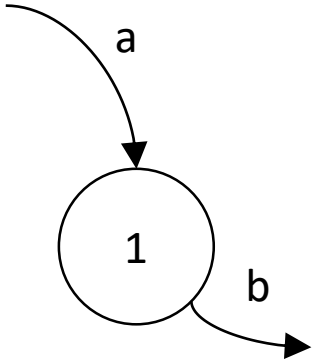
https://www.youtube.com/watch?v=hiMeEswjWuk

Functional Flow Diagram – YouTube Example

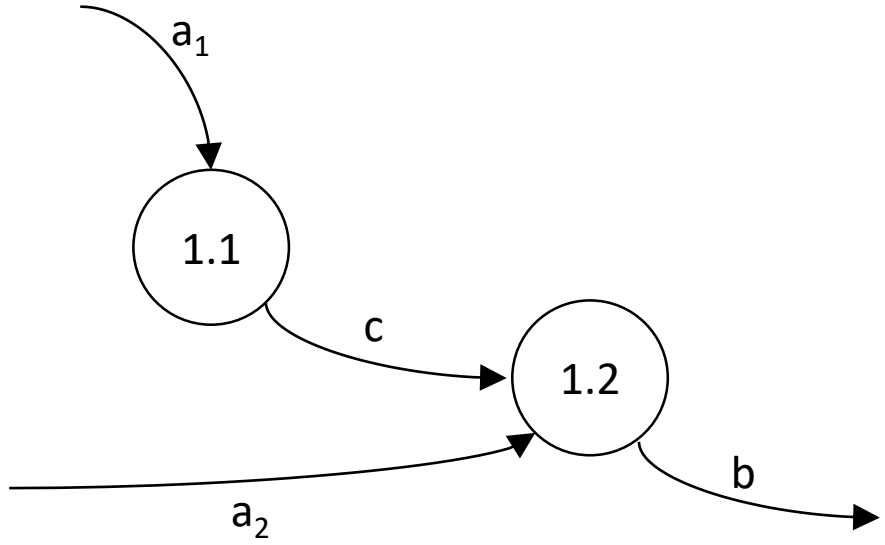


https://www.youtube.com/watch?v=hiMeEswjWuk

Balancing Flows



HIGHER LEVEL FFD



LOWER LEVEL FFD

Inputs and outputs need to balance between parent and child diagrams

$a = a_1 + a_2$

Functional Modelling

ADVANTAGES

- Shows relationships **between** functions
- Simple technique, which means that the whole team can be involved in the process
- Provides a common understanding of the system
- Customers can also get involved
- Oversights and errors can be detected early
- Identifies the external parts of the system

DISADVANTAGES

- Difficult to verify completeness
- Diagrams can get complicated if not broken down into separate levels properly
- **Functions and data are treated as separate**

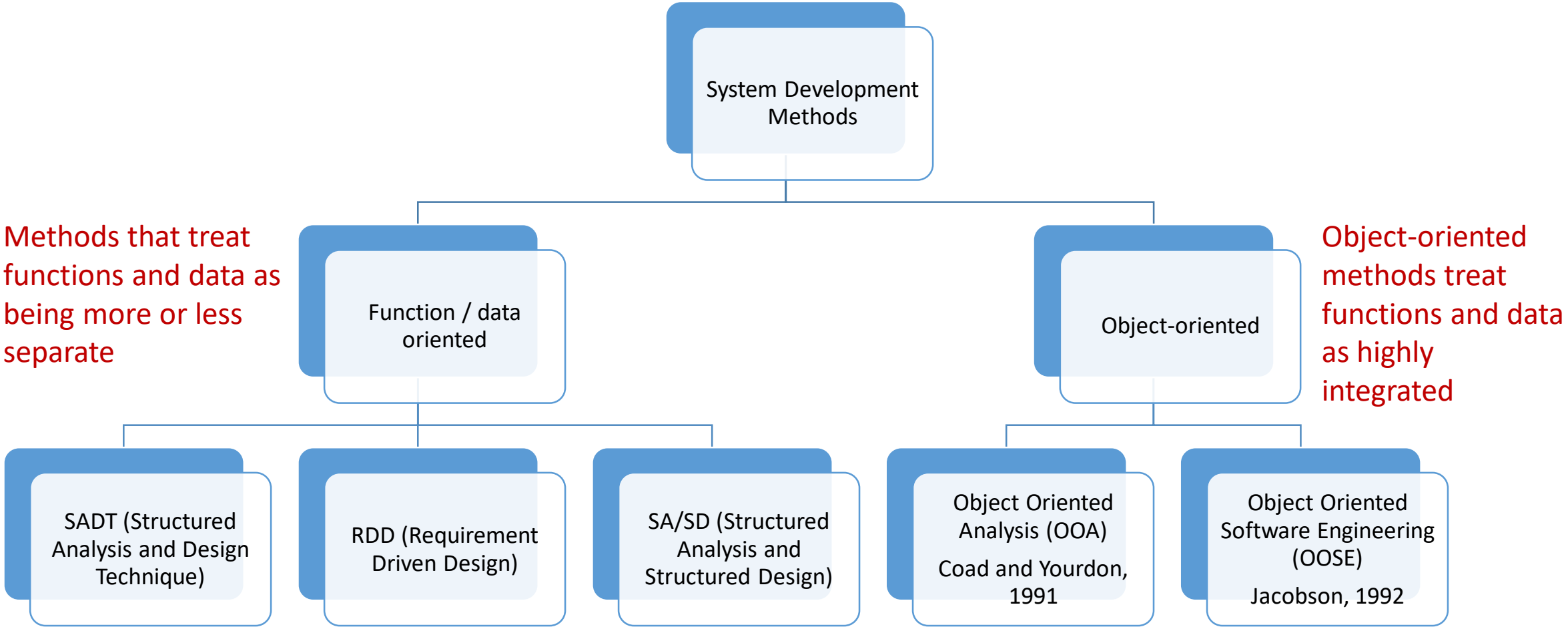
Group Exercise

- Work in groups
- Construct a high level FFD for an email system

(10 minutes)

System Development Methods

System Development Methods



Methods that treat functions and data as being more or less separate

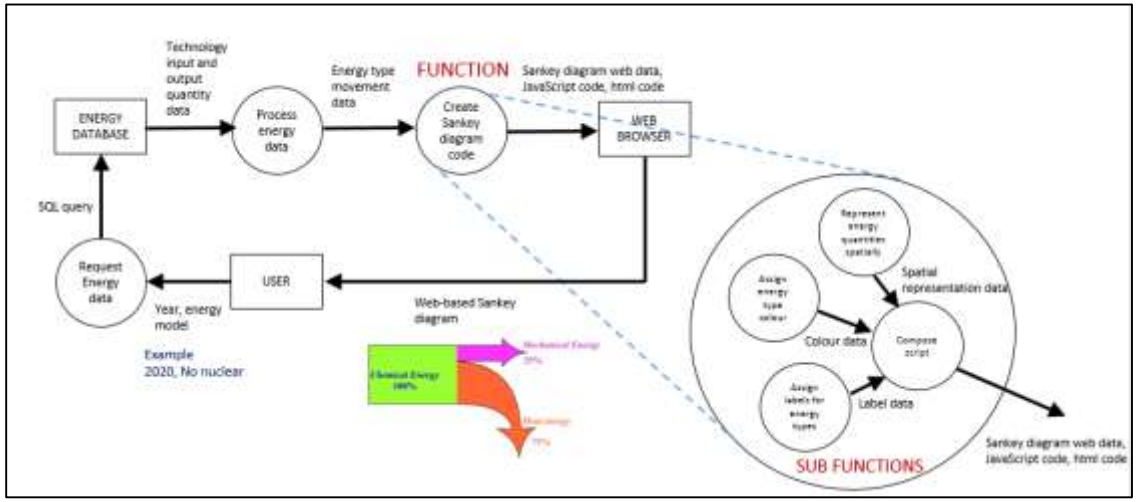
Object-oriented methods treat functions and data as highly integrated

TRADITIONAL SOFTWARE ENGINEERING METHODS

OBJECT-ORIENTED SOFTWARE ENGINEERING METHODS

Function / Data Development Methods

- These methods distinguish between functions and data
- Functions are seen as active, with a given behaviour; data is seen as a passive holder of information that is affected by functions
- System is thus typically broken down into functions, with data sent between those functions
- The design is based around how a certain behaviour will be carried out with functions eventually converted into source code



- However, the paradigm is an artefact of computing
- Assembly language divides instructions and data, and memory has separate spaces for instructions and data in modern processors
- High level languages tended to follow suit
- Object-orientation is an attempt to escape from this approach

Drawbacks of Separating Functions and Data

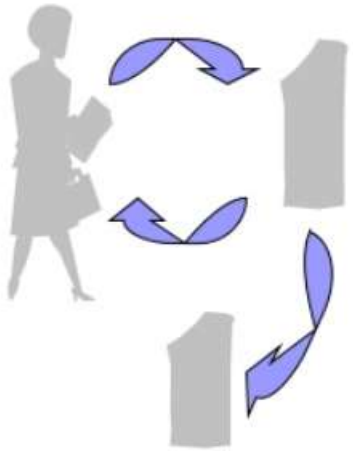
- A major problem is that, in general, all functions must know the data structure, which means that it has to be hard-coded into the function
- If we want to modify the data structure, we then have to modify all the functions relating to it
- Slight modifications to the data structure can have a major impact on the code, often making it unstable. This means systems developed using the functional / data method can be difficult to maintain

Drawbacks of Separating Functions and Data

- Also, requirements are often captured in natural language – what exists in the system and what it will do
- The function / data method separates everything into either data or functions, which is not natural.
- It requires a translation when we start to think about how the system will work. This is known as a **semantic gap** between the model and reality

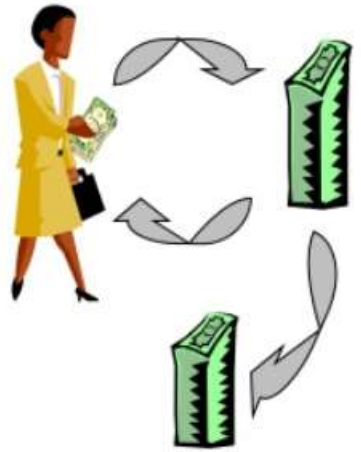
Functional Thinking vs OO Thinking

Functional



Withdraw, deposit, transfer

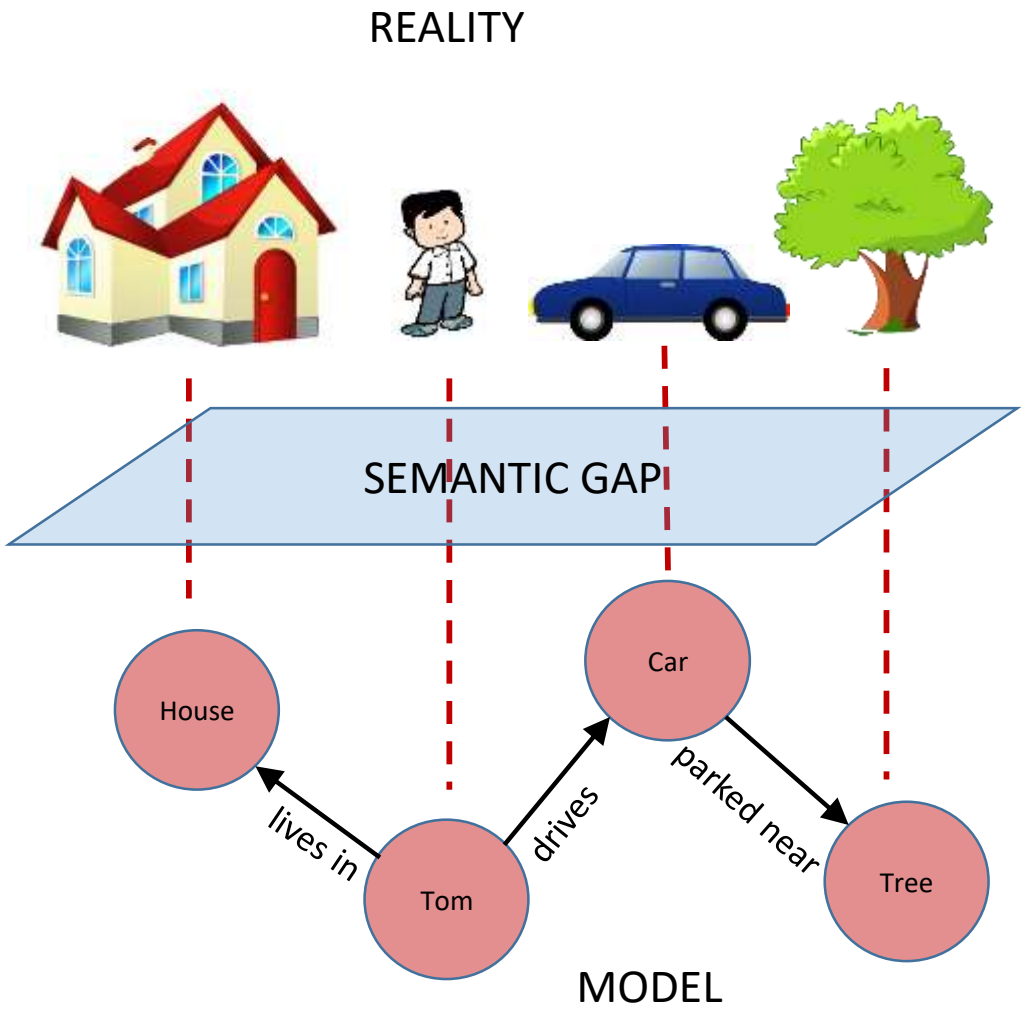
Object Oriented



Customer, money, account

Object Orientation

- A technique for system modelling where the system has a number of objects that interact. It is independent of any programming language
- A model designed using object-orientation is often easy to understand as it can be directly related to reality. Often there is only a small gap between reality and the model.



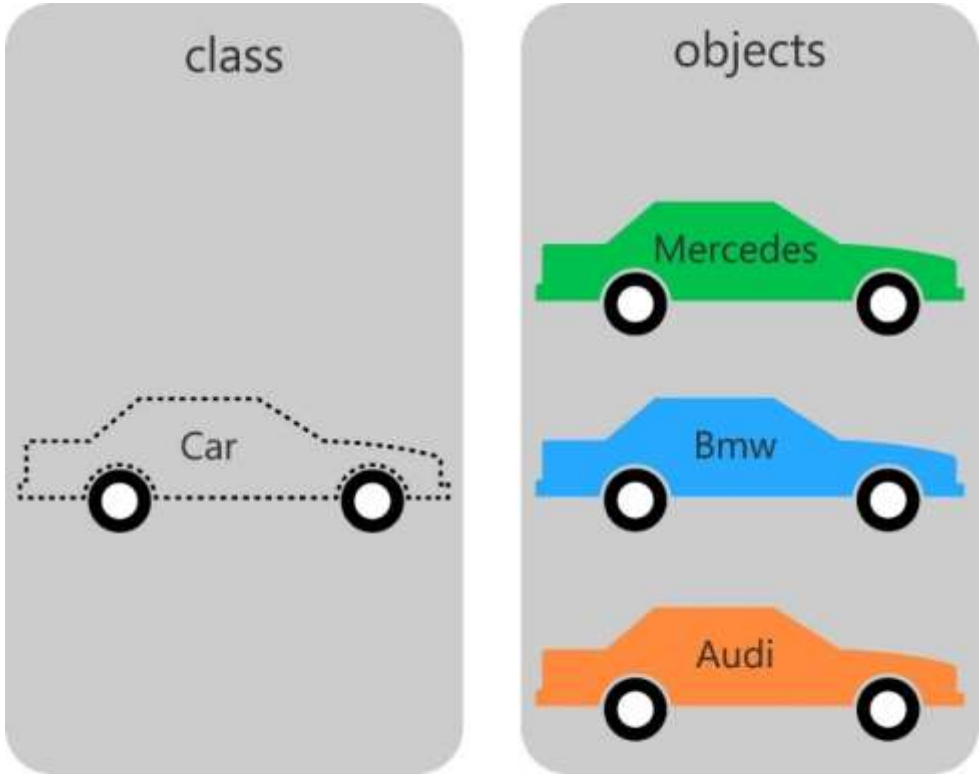
Addressing the Drawbacks of Functional Modelling

- The objects that exist in the system are usually stable and change very little
- When changes are needed they usually only affect one or a few objects, which means that, usually, only small changes are necessary
- If we wish to have a stable system we should consider what tends to change and then design according to this knowledge
- With object-orientation the requirements are modelled from the natural language
- What exists in the system is modelled as an **object** and what it **does** is modelled as a method of that object

Object-oriented Analysis

Object-oriented Analysis

- The purpose, as with all other analysis methods, is to understand the application in terms of the system's functional requirements
- The difference is the means of expression
- Function / data analysis considers the system's data and behaviour separately
- Object-oriented analysis combines them and regards them as integrated objects
- Object-oriented analysis consists of the following five steps:
  1. **Identifying the objects**
  2. **Organising the objects**
  3. **Describing how they interact**
  4. **Defining the operations on the objects (methods)**
  5. **Defining the objects internally (properties)**



1. Identifying the Objects

- The first step in OO analysis is to identify all the objects in the problem domain
- These can be found by examining all the **nouns** within this domain

king queen bishop knight rook pawn

board

player1

player2

2. Organising the Objects

- Different object types can be organised according to different criteria
- Examples:
  - Active / passive
  - Physical / conceptual
  - Temporary / permanent
  - Part / whole
  - Generic / specific
  - Private / public
  - Shared / non-shared

- Identifying which objects work together
- Deciding whether an object is part of another, e.g. a house can be composed of doors, windows, bricks etc.
- Deciding which objects are dependent on others

2. Organising the Objects

- Another way of organising them is to consider how similar they are to each other
- If they are similar they may belong to the same *class*

king queen bishop knight rook pawn

Sub-classes

Chess piece

Class

Class

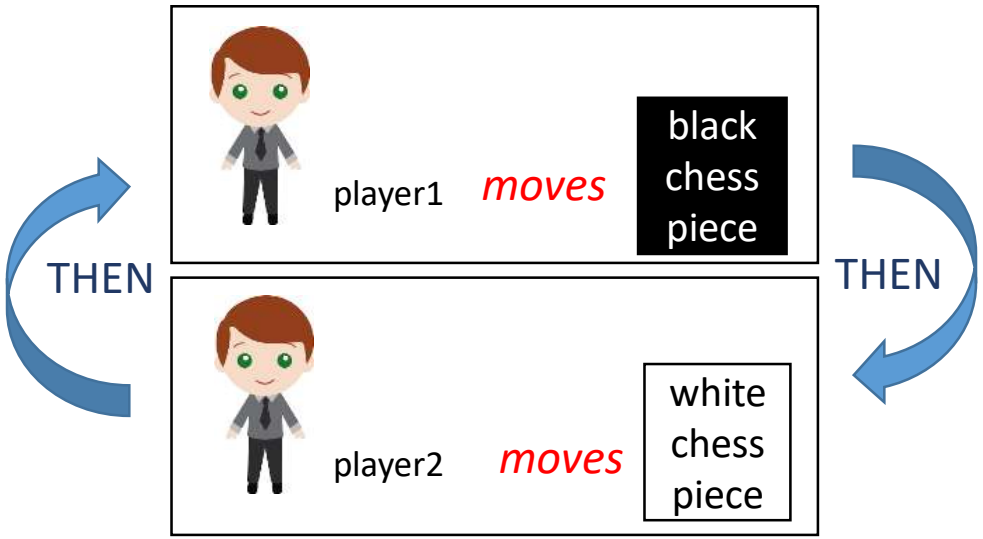
For example, the generic 'chess piece' is a physical game piece that is capable of moving on the chess board somehow and is either black or white in colour

Sub-classes

Inherit the characteristics of the class that they belong to, so each of them moves on the chess board and is either black or white in colour

3. Describing Object Interactions

- Next, we need to think about how the objects interact with each other.
- We can describe different scenarios or ***use cases*** in which the objects take part and communicate with each other
- This enables us to determine what the other objects expect from a given object



4. Operations on Objects

- We need to consider what can be done with each object in our system
- The operations could be primitive, for example, create, add, delete
- Or they could be more complex, for example, assembling a report based on information from other objects
- The operations we identify are called **methods**

king queen bishop knight rook pawn

In our chess example we identified that the class 'chess piece' has a method 'move'

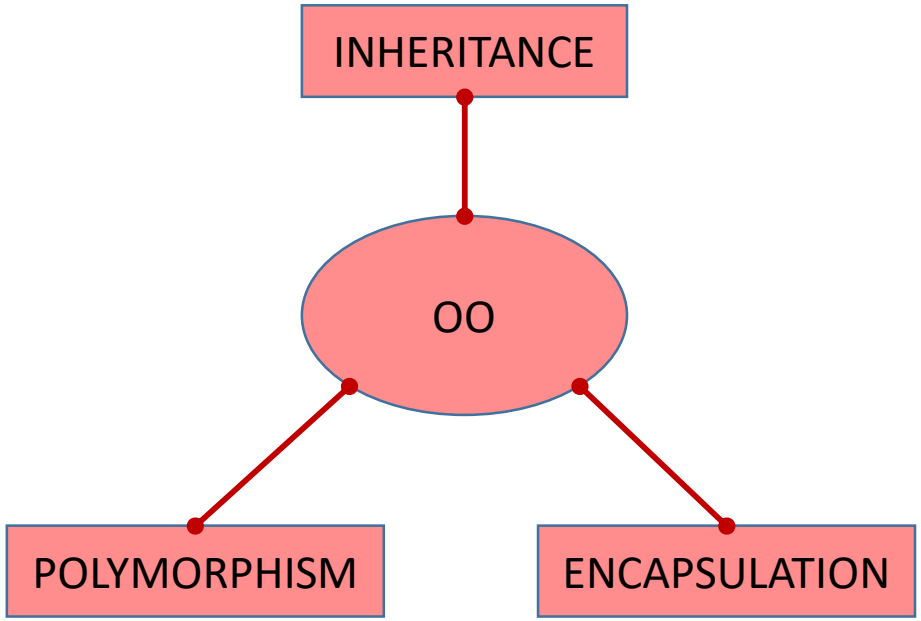
5. Defining Objects Internally

- This means defining the information that each object must hold
- To do this we assign properties to each class type

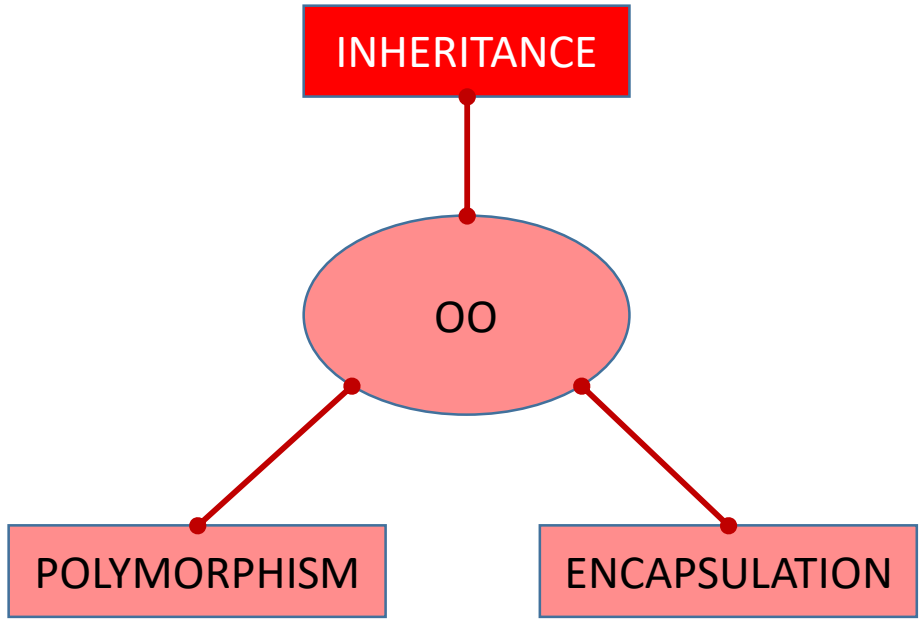
king queen bishop knight rook pawn

In our chess example we identified that the class ‘chess piece’ has property ‘colour’

Principles of Object-oriented Design



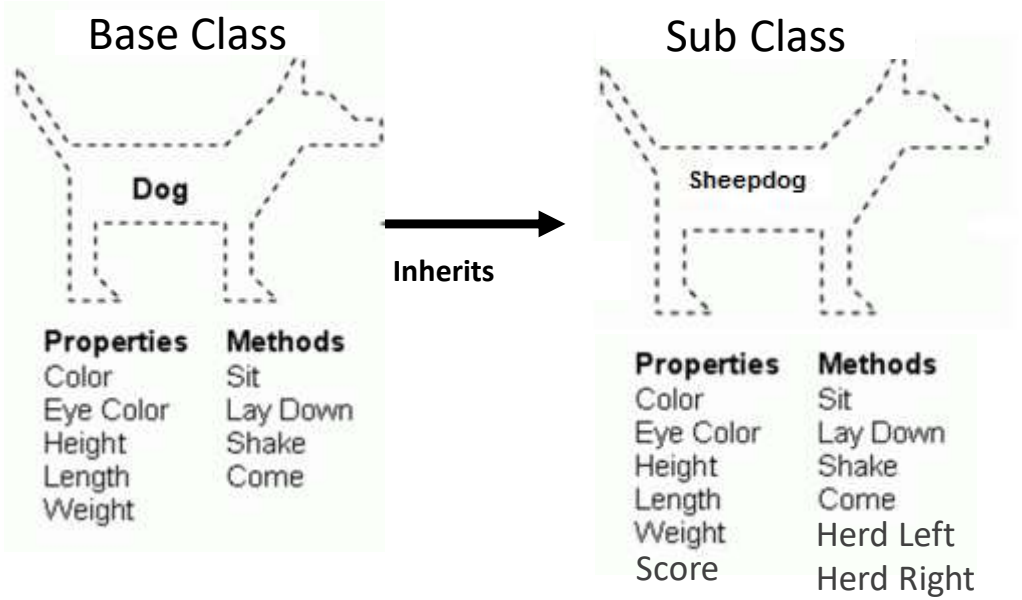
Inheritance



- When a sub-class inherits from another class it inherits all of its properties and methods
- In our chess example the sub-class bishop inherits all of the properties and methods of the base class ‘chess piece’, i.e. the bishop has a colour property and move method

Inheritance

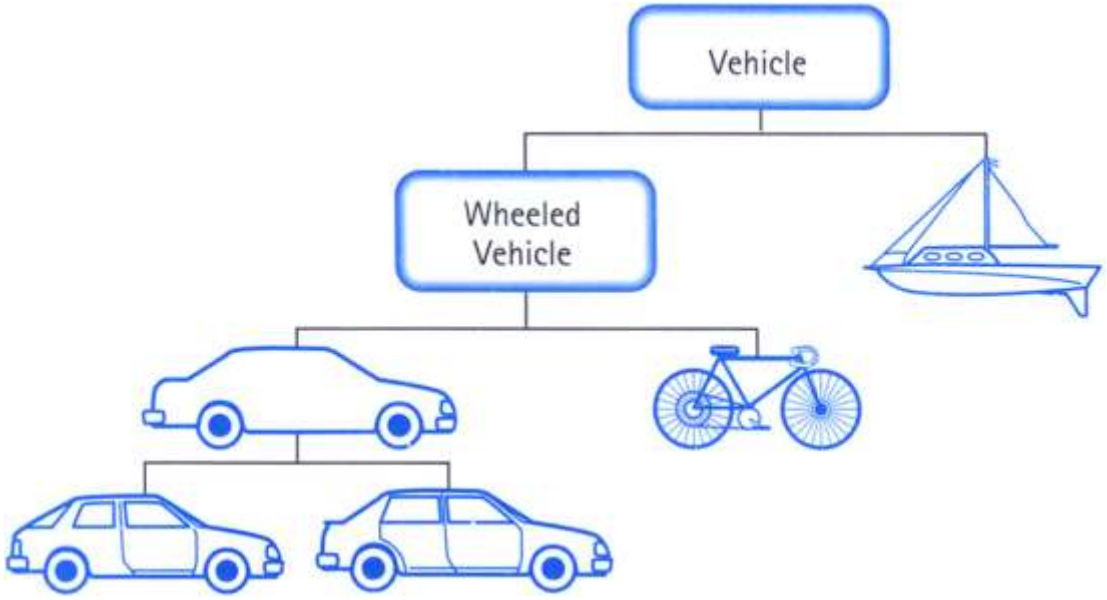
- You create a more specialised version of the class
- The sub class can add new properties and methods
- For example we could add a *Score* property to store the sheepdog's performance in sheep trials
- We could add methods for rounding up sheep.



- Inheritance has many advantages.
- What are they?

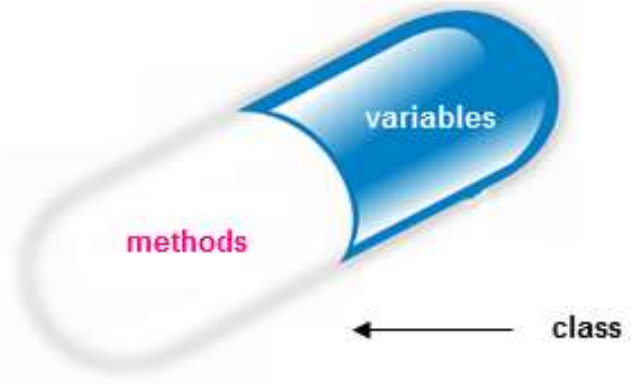
Inheritance - Advantages

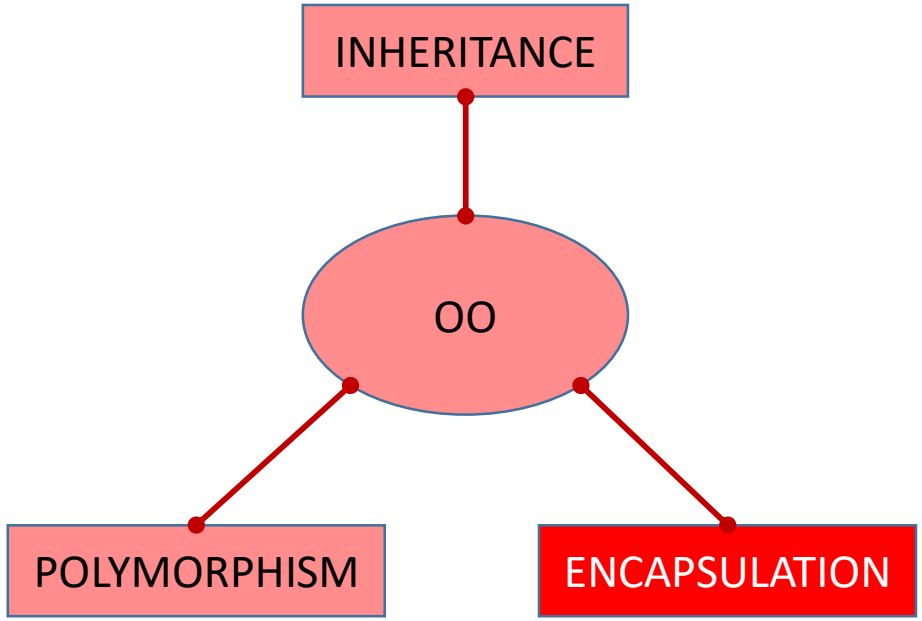
- Forces a more thorough data analysis
- Reduces development time
- Reduces code length
- Avoids redundancy
- Reduces errors as code is reused more
- Consequently saves money



Encapsulation

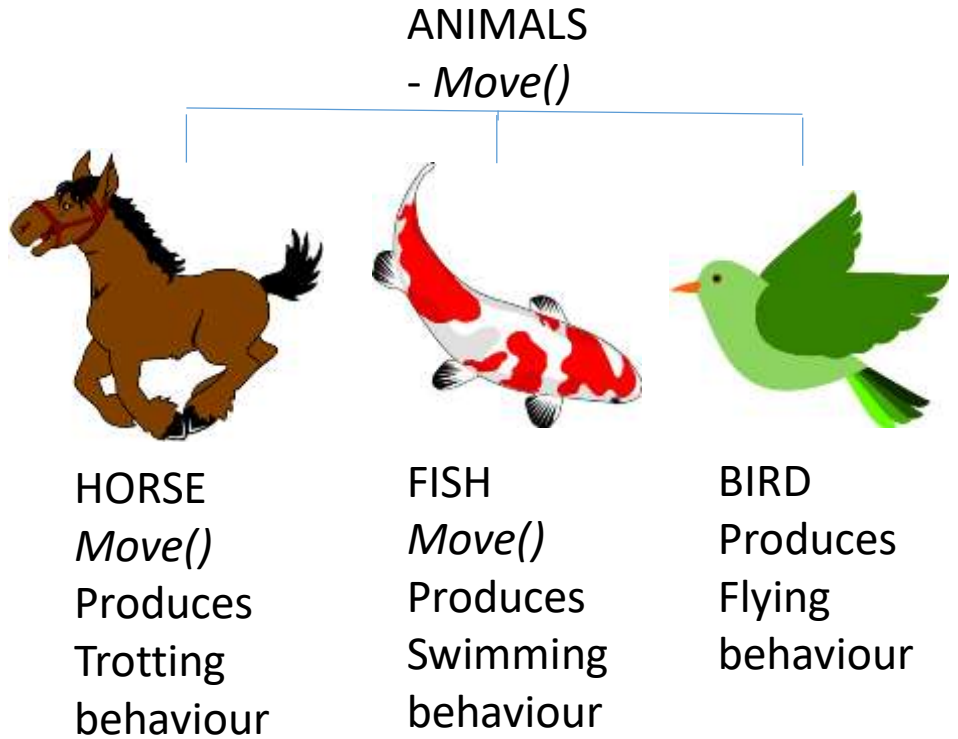
- We have seen that object-orientation binds together the data (attributes / properties) and the functions (methods) that manipulate it, within a class. This is known as encapsulation.
- Encapsulation keeps both the attributes and methods safe from outside interference and misuse leading to the important OO concept of *information hiding*.
- The *internal implementation* of a class is hidden from the rest of the program.
- Encapsulation thus helps to create code that is ***loosely coupled***.
- The ability of other objects to modify an object's state and behaviour is reduced because the details are hidden





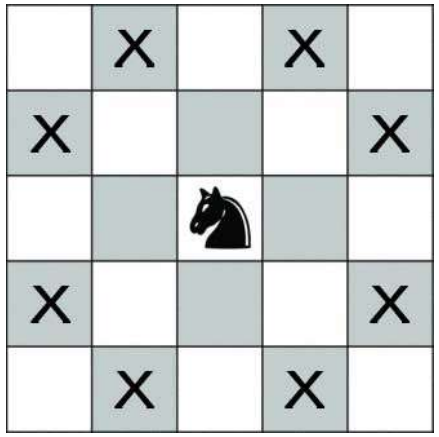
Polymorphism

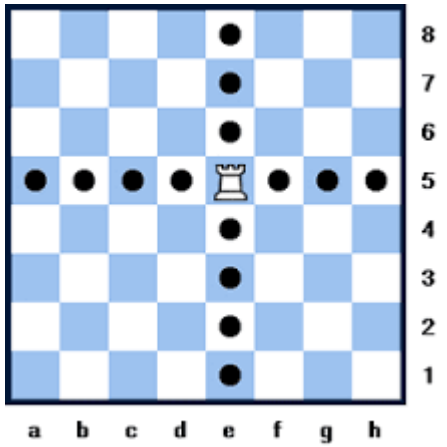
- Sub-classes can possess methods bearing the same name as those in the base class, but each may have different behaviours.
- As an example, let us assume there is a base class named Animals from which the subclasses Horse, Fish and Bird are derived.
- Let us also assume that the Animals class has a function named Move, which is inherited by all sub-classes mentioned.
- With polymorphism, each subclass may have its own way of implementing the function.
- So, for example, when the Move function is called in an object of the Horse class, the function might respond by displaying trotting on the screen.
- On the other hand, when the same function is called in an object of the Fish class, swimming might be displayed on the screen. In the case of a Bird object, it may be flying.



Polymorphism

king queen bishop knight rook pawn

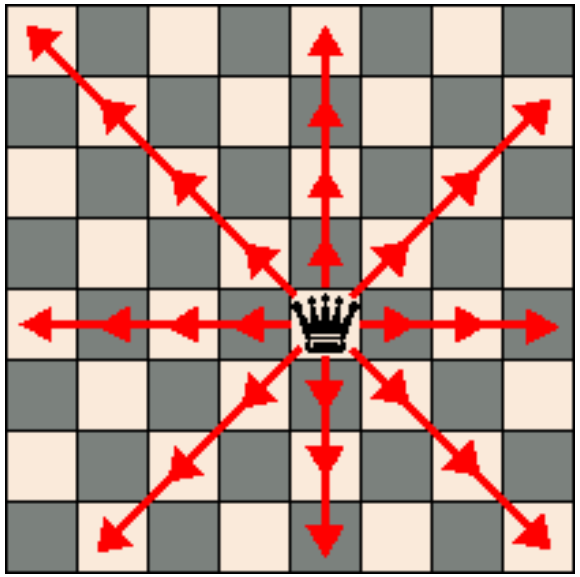




In our chess example we identified that the class 'chess piece' has sub-classes:

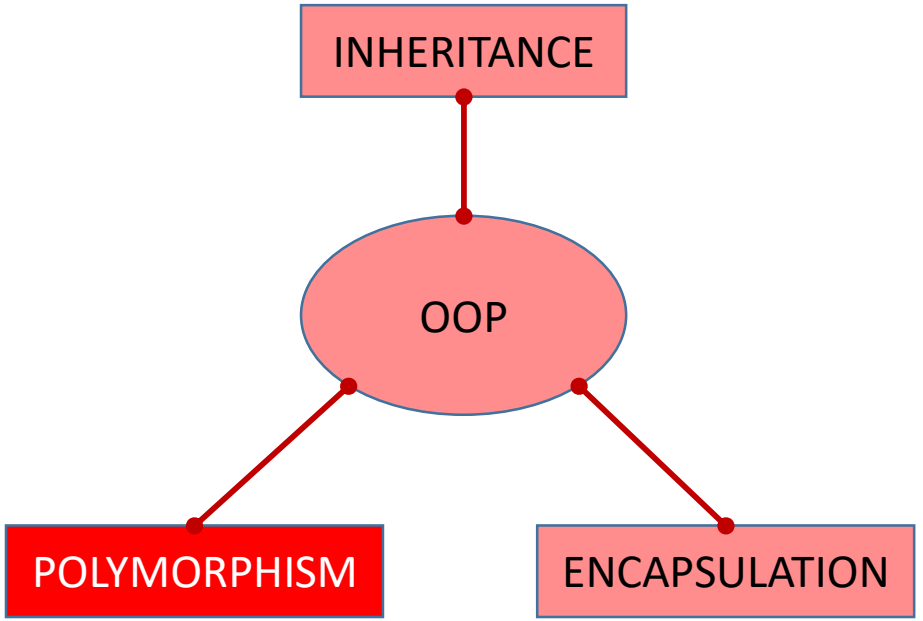
- King
- Queen
- Bishop
- Knight
- Rook
- Pawn

Each piece has the method 'move', but we are not restricted to making that method the same for each sub-class. For example, the queen moves in an entirely different way to the knight or rook



Polymorphism

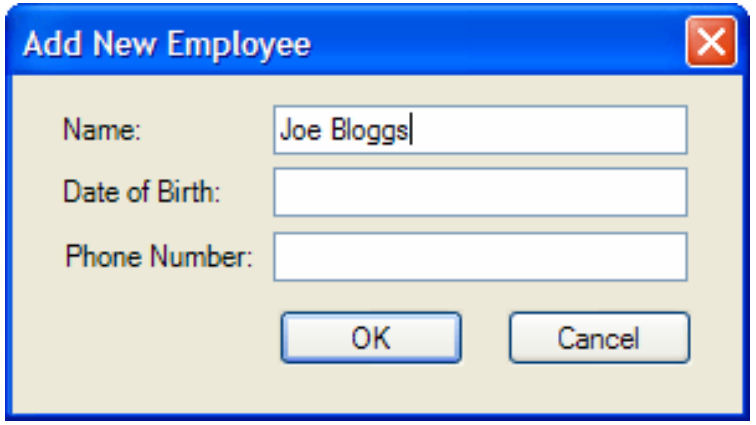
- In effect, polymorphism trims down the work of the developer.
- We can initially create a general class with all the attributes and behaviours necessary.
- When more specific subclasses are needed, with certain unique attributes and behaviours, the developer can simply alter code in the specific portions where the behaviours will differ.
- All other portions of the code can be left alone



Object-oriented Construction

Object Oriented Programming

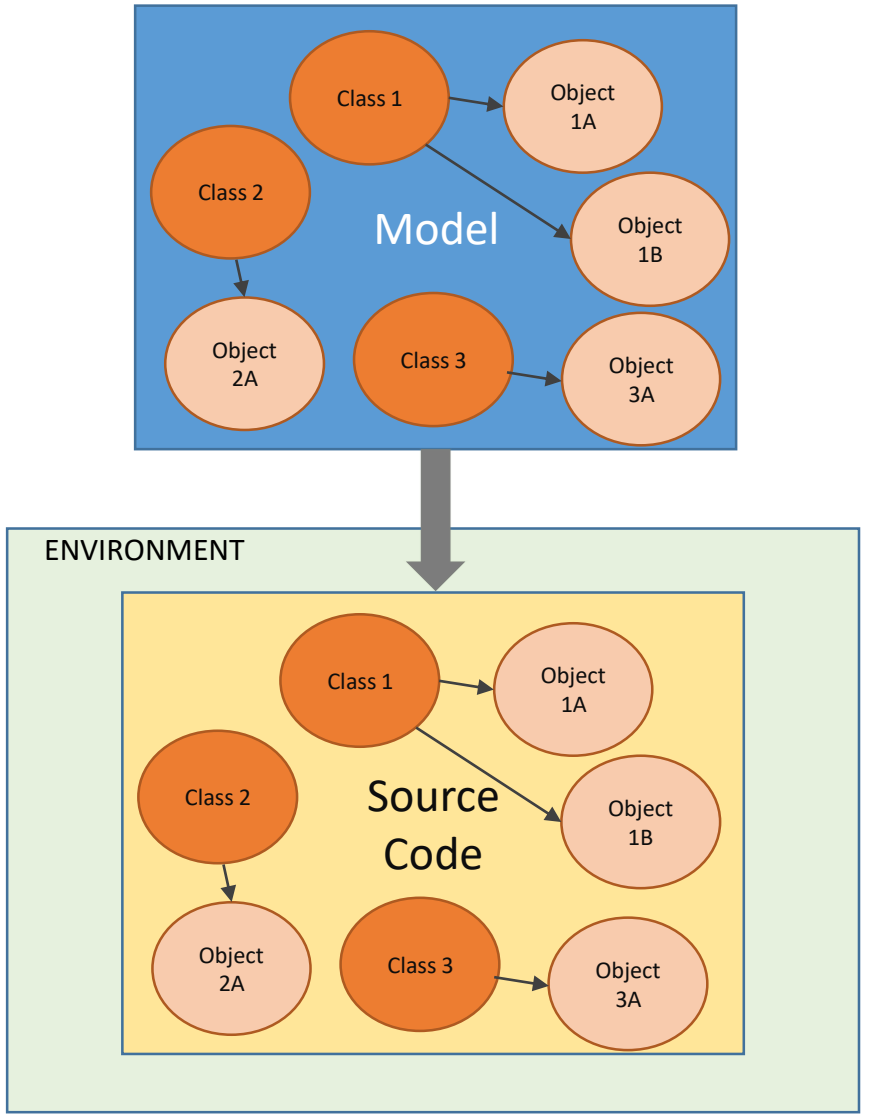
- Object-oriented programming (OOP) is a programming language model organized around **objects** rather than **functions** and the **logic** required for them.
- Historically, a program was viewed as a logical procedure that takes input data, processes it, and produces output data.
- The programming challenge was seen as **how to write the logic**, not **how to define the objects** in the system.
- Object-oriented programming takes the view that **what matters most are the objects** we want to manipulate, not the logic required to manipulate them.



- Examples of objects range from human beings (described by name, address, etc.) to buildings and floors (whose properties can be described and managed) down to the little widgets on a computer desktop (such as buttons and scroll bars).

Object Oriented Construction

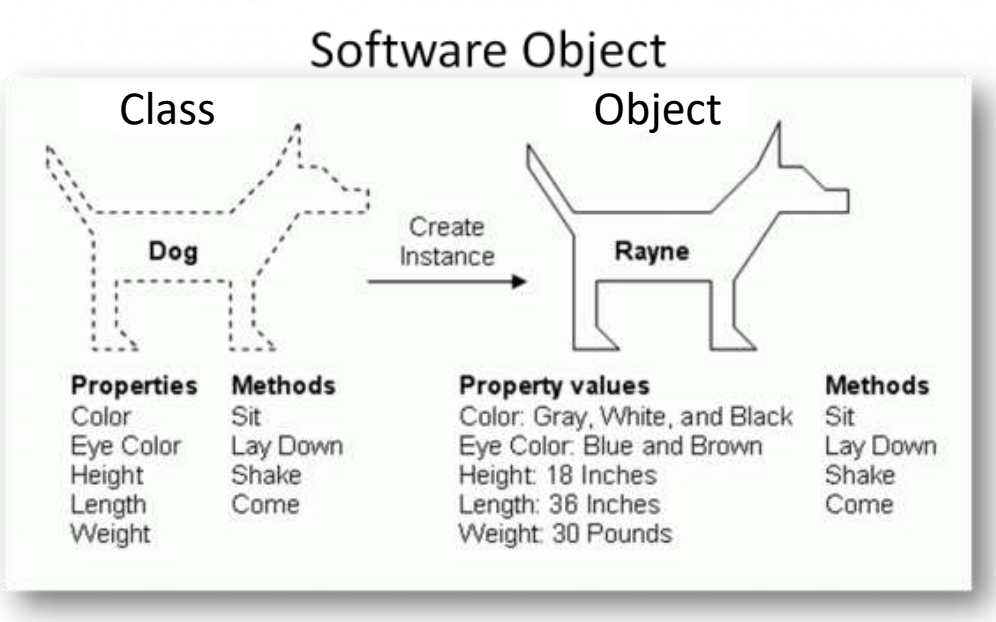
- Object-oriented construction involves turning the analysis model into source code, which is executed in the target environment
- The goal is that the classes and objects identified during the analysis should also be found within the code.
- This is known as **traceability**



Classes and Objects

- Class
  - Defines how to make an object of that type
  - Blueprint for an object
  - Has properties and methods

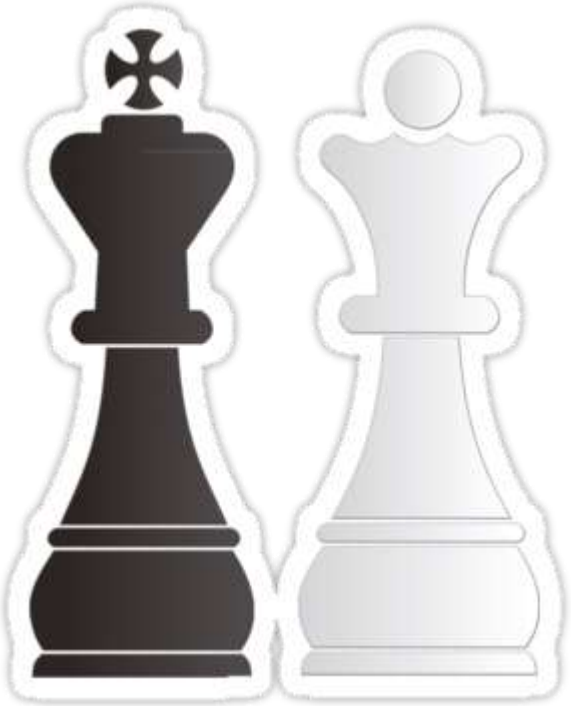
- Object
  - Instance of a class
  - Properties take values
  - These show the state of the object



Instantiation

In our chess example we identified that the class 'chess piece' has sub-classes:

- King
- Queen
- Bishop
- Knight
- Rook
- Pawn



We can create an instance of the sub-class King and call it *BlackKing*, and another instance of the sub-class and call it *WhiteKing*. We can set the 'colour' property of the black king to black and the colour property of the white king to 'white'

Steps in Object Oriented Modelling

1. Identify the objects – look for nouns in the system
2. Organise the objects into a class structure
3. Describe how the objects interact – think of possible use cases
4. Define the operations on the objects - methods
5. Define the objects internally - attributes

Group Exercise

- Work in groups to create an object-oriented model of one of the following systems:
  - A library system
  - A university application and enrolment system
  - A vehicle booking system
  - The system you are working on for Team project
  - Any other system that you find interesting

(20 Minutes)